

Meridian Service Desk — Triage & SOP Agent

An agentic AI system that drafts service-desk replies citing the procedure they came from, and cannot send any of them on its own.
Aman Mishra · IIM Udaipur · Building with Agentic AI, capstone · September 2026

Live: <paste your Render URL here>

Code: <https://github.com/AManMishra20/TicketAgent>

What this is

A service desk this size takes about 1,200 tickets a month, and most are repeats. The fix is usually written down already. That helps nobody if the engineer on shift has never read it.

Two agents work opposite ends of the same problem. **Agent A** reads an incoming ticket, checks the requester's support tier, searches the procedure library, and drafts a reply citing whichever procedure it used, by ID. **Agent B** takes the other case. When a ticket gets resolved that no procedure covered, it reads the thread to see what the engineer actually did, then drafts the procedure that was missing.

The setting is enterprise IT support: Jira for tickets, Confluence for procedures. Both are stand-ins, backed by a local ticket store and a folder of markdown, so nothing external can break on submission day. Each panel says so, instead of putting a green 'connected' badge over a SQLite file.

The core design decision

The approval gate is **a code boundary, not a prompt instruction**. Asking a model to check before it acts is a request, and requests can be argued out of, including by text sitting inside a ticket. Four layers make it structural instead:

1. Rule	Stated in CLAUDE.md and the system prompt. A request.
2. Gate	Runs in code once the model has spoken, reading the ticket and the requester record. It never consults the model's opinion.
3. Boundary	The model holds no write-capable tool at all. Its only terminal move produces a proposal object.
4. Hook	Refuses any code edit that would widen the tool surface.

Four conditions stop a draft: a request for access or permissions, critical severity, a Gold-tier requester, and no procedure covering the ticket. An unknown requester counts as Gold, since the safe assumption there is the expensive one. The last condition matters most. When the agent reports that it cannot answer, the system is working: the ticket goes to Agent B to write what was missing.

Air Canada's chatbot invented a refund policy and a tribunal held the airline liable. Citations here are checked in code after the model speaks: an ID that does not resolve is stripped and the proposal downgraded.

How it works

Read	The agent chooses its own inputs: ticket, requester record, procedure library. Nothing is pasted in for it.
Score	Computed in code, not the model's estimate of its own confidence. The reviewer sees the number the agent saw.
Propose	The model's only terminal move: a structured object, never an action. A second instance writes the reply from three inputs.
Hold	The gate decides again, independently, and stamps HOLD where a condition applies.
Approve	A person decides. Nothing else in the system writes anything.

Checking it yourself

Everything runs offline without an API key, so the figures opposite can be checked rather than trusted:

```
python -m src.ticket_agent.seed --seed 42
pytest -q
uvicorn src.ticket_agent.web.app:app
```

The corpus comes from a fixed seed: 53 tickets, 6 procedures, and five defects planted on purpose, because real queues arrive carrying all five.

Using the website

- 1. Overview.** Three panels: Jira, Confluence, and the agent itself. Press **Activate**. The agent panel turns green and starts picking up tickets by itself.
- 2. Review queue.** Every row is something the agent wants to do and has not done. A red spine means it stopped. Green means it is ready to go.
- 3. Open a red row.** It carries a HOLD: HUMAN REVIEW stamp and names the condition that stopped it. Try approving with the note field empty. **It refuses.** That is the whole point of the thing.
- 4. Open a green row.** The right column shows the matched procedure and the score it matched at. Edit the reply if it needs it, then approve. Only now does it reach the ticket.
- 5. Back to Overview.** Your decision lands in the counts. Edits are stored beside the agent's original wording, so the same correction recurring points at a missing rule, not three unrelated mistakes.

Tickets and procedures are browsable from the left rail at any time. **Deactivate** stops the agent.

Measured results

Retrieval precision@1 (38 covered tickets)	100%
Uncovered tickets wrongly matched	0
Open tickets held for a human / ready	29 / 10
Correct matches routed to a human anyway	5 of 38
Automated tests, offline, no API key	83 passing

The 0.37 threshold was calibrated against known ground truth rather than picked by feel. It costs 13% recall and buys zero false citations. That asymmetry is deliberate: a false refusal wastes a reviewer thirty seconds, while a false citation is a policy stated in writing to someone who will act on it.

Declaration of AI use

This project was built with Claude (Anthropic) acting as a coding agent. The extent of that is substantial, and worth stating precisely.

What the AI produced: essentially all of the source code, the tests, the interface and its styling, the repository documentation, and the text on this page. It also proposed the retrieval-scoring approach and calibrated the 0.37 threshold against the seeded corpus. It caught a thread-safety bug in the background worker that I would not have found until it crashed something.

What I brought: the problem itself, from service-desk work at TCS; the constraint that the approval gate had to be structural rather than a line in a prompt, which shaped most of what followed; the decisions the agent stopped to ask about, including the data backend, the deployment target and how the stand-in systems should be labelled; review at each stage; and the deployment.

Anyone can check the claims above against the repository. The test suite runs without an API key.

One limitation, stated plainly: every figure here comes from demo mode, where a fixed decision procedure stands in for the language model over the same retrieval results. The safety architecture, the retrieval numbers and the interface are real. Live-model accuracy and token cost are not measured at all, and docs/EVIDENCE.md records them as unmeasured instead of estimating them.